

TOP with ORDER BY Semantics

#sql-server #semantics #query-behavior

The Common Misconception

“ORDER BY is meaningless on a SELECT INTO.” This is half true and dangerously misleading when TOP is involved.

Without TOP: ORDER BY Is Meaningless

```
-- ORDER BY is truly meaningless here. SQL Server may ignore it.  
Select OrderId, Status  
Into #TempOrders  
From OeOrder  
Order By OrderId  
-- The rows in #TempOrders have NO guaranteed order.
```

SQL Server is free to ignore the ORDER BY because a table (even a temp table) is an unordered set by definition. The optimizer may honor the ORDER BY if a suitable index makes it free, but it isn't guaranteed.

With TOP: ORDER BY Determines Which Rows

```
-- ORDER BY is CRITICAL here. It determines which rows TOP selects.  
Select Top (100)  
    OrderId, Status  
Into #TopOrders  
From OeOrder  
Order By OrderId Asc  
-- This selects the 100 LOWEST OrderIds. Different ORDER BY = different rows.
```

When TOP is present, the ORDER BY defines the selection criteria. It is the rule for which rows qualify for the top N. Changing the ORDER BY changes which rows are returned. This is not about the physical order of the result. It is about which rows exist in the result at all.

Why This Matters for Performance Reviews

During a code review, it's tempting to flag ORDER BY on SELECT INTO as wasteful. But you must check for TOP first:

```
-- From lsp_ImgGetListOfTopXImagesToMove:  
Select  
    Top (Select Convert(Int, [Value]) From vCfgSystemParamVal Where ...)   
    i.Id, i.ContentCode  
From vImgImage i  
-- ... joins ...  
Where -- ... filters ...  
Order By i.Id Asc -- NOT meaningless. Determines which images get picked.
```

Removing this ORDER BY would change the proc's behavior. Instead of picking the oldest images (lowest IDs), it would pick whichever images the optimizer happens to return first, which is nondeterministic and could change between executions.

The Performance Implication

TOP combined with ORDER BY creates a rowgoal optimization. The optimizer knows it only needs N rows in a specific order, so it may:

1. Use an ordered index scan. If an index on the ORDER BY column exists, it scans in order and stops after N rows. Very efficient.
2. Use a Top N Sort. If no suitable index exists, it reads all qualifying rows, sorts them, and returns the top N. The sort uses a special “top N sort” operator that only keeps N rows in memory, discarding others as they arrive.

When the TOP value comes from a subquery (as in our proc), the optimizer doesn't know N at compile time and can't optimize the rowgoal as aggressively. Pulling the value into a local variable first helps:

```
Declare @MaxImages Int
Select @MaxImages = Convert(Int, [Value])
From vCfgSystemParamVal
Where Section = 'Operation'
And Parameter = 'MaxNumberOfImagesToPullInASingleRunFromBgWk'

-- Now the optimizer sniffs @MaxImages and can set rowgoal
Select Top (@MaxImages) ...
Order By i.Id Asc
```

Related Concepts

- **Parameter Sniffing**: the TOP value from a variable is sniffed at compile time for rowgoal estimation.
- **LEFT JOIN OR Anti-Pattern**: TOP can't optimize rowgoal effectively when the underlying query has OR across join paths.